

Catatan dari penulis:

Hasil pemikiran tidak dipublikasikan. Terdokumentasi di Perpustakaan Departemen Ilmu Komputer IPB, untuk pembimbingan tugas akhir mahasiswa.
endang_pg@apps.ipb.ac.id

CONVOLUTIONAL NEURAL NETWORK KONSEP, PENERAPAN, DAN IMPLEMENTASI DENGAN CONTOH EKPERIMEN

ENDANG PURNAMA GIRI

Staf Pengajar: Departemen Ilmu Komputer FMIPA IPB



**DEPARTEMEN ILMU KOMPUTER
INSTITUT PERTANIAN BOGOR
BOGOR
2021**

RINGKASAN

ENDANG PURNAMA GIRI. Departemen Ilmu Komputer IPB, Fakultas Matematika dan Ilmu Pengetahuan Alam, IPB.

Melalui kajian studi literatur ditemukan fakta bahwa banyak kasus klasifikasi bagi data akustik ataupun data *image* tercatat memperoleh keberhasilan ketika digunakan pendekatan *deep learning* khususnya arsitektur Convolutional Neural Network (CNN). Arsitektur CNN pada penerapannya bertindak sebagai modul ekstraksi *feature* sekaligus juga sebagai modul klasifier. Berdasarkan hasil tersebut maka pendekatan *deep learning* diperkirakan dapat mengekstraksi *feature* secara mandiri bagi kasus klasifikasi kematangan dan tingkat kemanisan buah melon. Pada laporan teknis ini difokuskan membahas tentang *state-of-the-art* teknik klasifikasi dan ekstraksi *feature* berbasis akustik yang kemudian dilanjutkan dengan upaya pengkajian penerapan pendekatan teknik-teknik tersebut sedemikian sehingga dapat diintegrasikan pada arsitektur CNN yang ditujukan untuk pengembangan model klasifikasi tingkat kematangan buah melon.

Kata kunci: ekstraksi *feature* akustik, *deep learning*, CNN, melon, klasifikasi

Catatan dari penulis:

Hasil pemikiran tidak dipublikasikan. Terdokumentasi di Perpustakaan Departemen Ilmu Komputer IPB, untuk pembimbingan tugas akhir mahasiswa. endang_pg@apps.ipb.ac.id

DAFTAR ISI

DAFTAR TABEL	vi
DAFTAR GAMBAR	vi
DAFTAR LAMPIRAN	vi
1 CONVOLUTIONAL NEURAL NETWORK (CNN)	1
<i>Machine learning</i> dan <i>Deep Learning</i>	1
Algoritme CNN	3
Kelebihan CNN Dibandingkan <i>Neural Network</i>	8
Formulasi Matematika untuk CNN	10
2 PENERAPAN CNN UNTUK BERAGAM KASUS KLASIFIKASI	11
Teknik CNN pada Klasifikasi Berbasis <i>Feature</i> Akustik	11
Teknik CNN pada Klasifikasi Berbasis <i>Feature</i> Visual	12
3 IMPLEMENTASI CNN DENGAN CONTOH EKSPERIMEN	13
Contoh Data Eksperimen	14
Arsitektur CNN yang Digunakan	15
Hasil Eksperimen	16
4 KESIMPULAN	17
DAFTAR PUSTAKA	17

DAFTAR GAMBAR

Arsitektur machine learning vs deep learning (Dey 2018)	2
Arsitektur CNN pada MNIST (Le Cun <i>et al.</i> 1998)	4
Ilustrasi <i>convolution</i>	5
Ilustrasi <i>subsampling</i>	6
Ilustrasi <i>padding</i>	7
Ilustrasi <i>stride</i>	7
Ilustrasi <i>sparse connectivity</i>	8
Ilustrasi <i>parameter sharing</i>	9

1 CONVOLUTIONAL NEURAL NETWORK (CNN)

Dalam *machine learning*, CNN merupakan teknik yang termasuk pada *feed-forward neural network*. Pada arsitektur CNN setiap individu *neuron* disusun sedemikian sehingga dapat memberikan respon terhadap *region* yang saling *overlapping* pada suatu area visual (Ciresan 2011). CNN merupakan *hierarchical neural networks* yang tersusun atas beberapa *convolutional layer* dan beberapa *subsampling layer*. Pada pemanfaatannya CNN dapat divariasikan pada bagian jenis dan jumlah dari *convolutional layer* dan *subsampling layer* yang dimilikinya.

Machine learning dan Deep Learning

Deep learning mengacu kepada suatu teknik *machine learning* yang menggunakan banyak *layer* dari tahapan pemrosesan informasi dalam arsitektur yang berhirarki. Pada *deep learning* arsitektur berhirarki tersebut ditujukan untuk mengeksplorasi pola klasifikasi untuk mendapatkan *feature* sebagai representasi aksi pembelajaran (Li 2014). Teknik *Deep learning* merupakan area penelitian yang ber-*intersection* dengan *neural network*, *graphical modeling*, *optimization*, pengenalan pola, dan pemrosesan sinyal. Banyak penelitian sebelumnya yang telah mencatat keberhasilan penerapan *deep learning* dalam berbagai bidang yang meliputi *computer vision*, pengenalan *phonetic*, pengenalan suara dalam percakapan, *feature coding* bagi suara dan *image*, pengenalan tulisan tangan, pengenalan objek secara visual, *information retrieval*, sampai dengan analisis molekul yang ditujukan dalam upaya penemuan obat baru bagi bidang medis. Tiga alasan utama yang menjadikan *deep learning* saat ini semakin populer adalah (i) peningkatan kemampuan pemrosesan komputer yang drastis (terutama melibatkan GPU), (ii) semakin murahnya harga *hardware* dengan kemampuan komputasi yang baik, dan (iii) semakin mutakhirnya penelitian di bidang *machine learning* dan pemrosesan sinyal ataupun informasi.

Deep learning merupakan pengembangan dari teknik pembelajaran sebelumnya yang menggunakan arsitektur pembelajaran sederhana (berarsitektur dangkal). Pada arsitektur pembelajaran sederhana umumnya menggunakan *layer* tunggal bagi transformasi *feature* nonlinier dan tidak memiliki *multiple-layers* nonlinier *feature* yang adaptif. Beberapa contoh teknik konvensional yang memiliki arsitektur sederhana yang umum digunakan di antaranya adalah *Gaussian Mixture Models* (GMMs), *Hidden Markov Models* (HMMs), *Conditional Random Fields* (CRFs), *Maximum Entropy* (MaxEnt) *Models*, *Support Vector Machines* (SVM), *Logistic Regression*, *Kernel Regression*, dan *Multi-Layer Perceptron* (MLP) *Neural Network* dengan *hidden layer* tunggal termasuk di dalamnya *Extreme Learning Machine*.

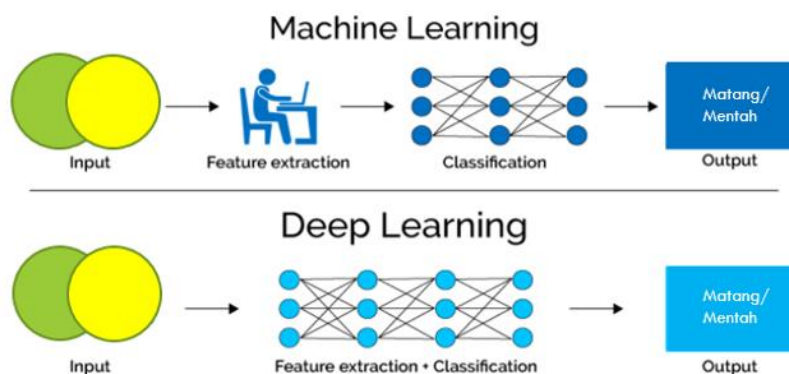
Teknik-teknik dengan arsitektur dangkal tersebut telah terbukti secara efektif dapat memecahkan banyak permasalahan yang *well-constrained*. Namun teknik-teknik tersebut akan memiliki keterbatasan ketika memodelkan dan bahkan kesulitan ketika merepresentasikan permasalahan kompleks dalam aplikasi dunia nyata. Salah satu contohnya adalah menginterpretasikan sinyal alami berupa suara manusia. Pada mekanisme pemrosesan informasi berupa suara manusia dibutuhkan

arsitektur pemrosesan yang mendalam (*deep*) untuk mengekstraksi struktur dan membangun representasi internal dari *input-input* sensorik yang kaya dan beragam. Dari data asli yang berupa gelombang suara dibutuhkan *layer* yang terstruktur dan berhirarki untuk mentransformasikan data dalam level gelombang ke level *linguistic* yang selanjutnya dapat digunakan sebagai landasan untuk kebutuhan membentuk persepsi.

Pemrosesan informasi dan sistem pembelajaran pada algoritme *deep learning* tersusun atas banyak *layer* dari tahapan pemrosesan nonlinier. Keluaran dari setiap *layer* akan diumpangkan sebagai masukan bagi *layer* yang lebih tinggi. Dua properti kunci kesuksesan teknik *deep learning* adalah: (1) sifat generatif dari model yang umumnya membutuhkan penambahan bagian *top layer* untuk melakukan fungsi diskriminatif misalkan dalam suatu tugas klasifikasi. (2) Tahapan *unsupervised pre-training* yang membuat efektifnya pemanfaatan data tanpa label ukuran besar bagi tujuan ekstraksi struktur dan sifat *regularities* dari *feature* data masukan.

Didasarkan kepada tujuan penggunaannya apakah bagi sintesis ataupun klasifikasi maka secara umum *deep learning* dapat dikategorikan menjadi 3 kelas yaitu (Li 2014):

1. *Generative deep architectures*, ditujukan untuk mengidentifikasi sifat korelasi tinggi dari data yang diamati untuk tujuan analisis ataupun sintesis, dan atau mengkarakterisasi *joint statistical distributions* dari data serta asosiasinya terhadap kelas.
2. *Discriminative deep architectures*, merupakan arsitektur yang secara langsung memiliki kemampuan melakukan aksi diskriminatif bagi klasifikasi terhadap pola.
3. *Hybrid deep architecture*, adalah arsitektur yang tujuan utamanya adalah untuk kebutuhan diskriminatif akan tetapi dalam pencapaiannya dibantu secara signifikan oleh arsitektur generatif. Arsitektur generatif membantu pada tahapan optimasi dan regularisasi serta penentuan kriteria dari parameter yang akan digunakan bagi aksi diskriminatif yang ingin dibentuk.



Gambar 1 Arsitektur machine learning vs deep learning (Dey 2018)

Esensi dari teknik *deep learning* adalah menentukan *feature* ataupun representasi dari data yang diobservasi secara hirarki. Pada prosesnya *feature* pada level yang lebih tinggi akan didefinisikan oleh *feature* dari level yang lebih rendah.

Berbeda dengan *machine learning* klasik yang secara jelas memisahkan antara modul ekstraksi *feature* terhadap modul klasifikasi. Pada arsitektur *deep learning* terutama yang berarsitektur diskriminatif, maka arsitektur *deeplearning* akan bertindak sebagai tahapan ekstraksi *feature* sekaligus juga sebagai tahapan klasifikasi (Gambar 12).

Arsitektur generatif (*generative deep architectures*) sering kali diasosiasikan dengan *unsupervised feature learning*. Ketika ingin menerapkan arsitektur degeneratif untuk mengenali pola maka konsep kuncinya adalah *pre-training* (pra pelatihan). Pada *pre-training* proses pembelajaran pada *lower-layer* tidak bergantung kepada semua *layer* yang ada di atasnya. Akan tetapi proses pembelajaran setiap *layer* dilakukan secara *greedy* bertahap *layer-by-layer* dengan urutan dari *layer* bawah menuju *layer* atas. Skenario inilah yang menjadi inti dari *pre-training* sebelum pada tahapan pembelajaran (pelatihan) berikutnya yang akan melibatkan seluruh *layer* secara bersamaan. Beberapa teknik *deep learning* yang merupakan *generative deep architecture* di antaranya adalah *autoencoder*, *deep Boltzman machine* (DBM), *Restricted Boltzman Machine* (RBM), *sum-product network* (SPN), dan *Recurrent Neural Network* (RNN).

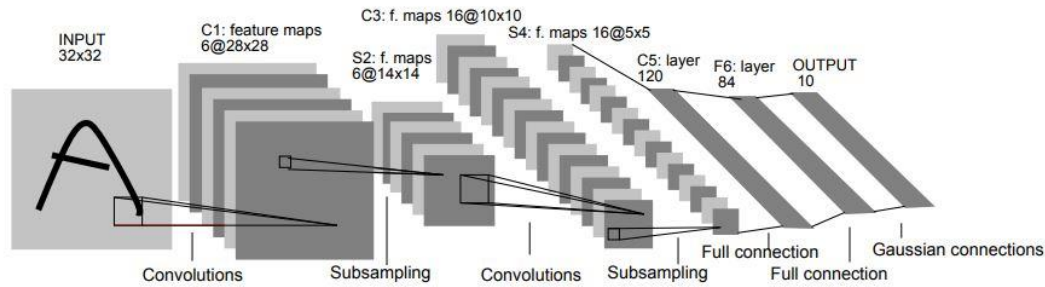
Berbeda dengan arsitektur generatif yang lebih kepada merupakan proses persiapan bagi aksi klasifikasi maka pada arsitektur diskriminatif (*discriminative deep architectures*) arsitekturnya telah juga memiliki modul untuk melakukan aksi klasifikasi itu sendiri. Salah satu contoh teknik yang merupakan arsitektur diskriminatif adalah *deep-structured CRF*. Berbeda dengan CRF konvensional yang memiliki arsitektur sederhana, pada *deep-structured CRF* *output* dari setiap *layer* CRF yang lebih rendah akan ditumpuk bersama dengan data *input* asli bagi *layer* yang lebih tinggi. Contoh lain yang tekniknya hampir serupa dikenal dengan *Deep Stacking Network* (DSN). Contoh arsitektur diskriminatif lainnya yang banyak digunakan dan memberikan kesuksesan di berbagai bidang klasifikasi adalah *Convolutional Neural Network* (CNN). Pada setiap modul CNN terdiri atas *layer* konvolusi (*convolutional*) dan *pooling layer*. Modul-modul pada CNN ini seringkali ditumpuk antara satu sama lain untuk mendapatkan *deep model*. *Layer* konvolusi dapat berbagi nilai bobot sedangkan *pooling layer* melakukan *subsample* terhadap *output* dari *layer* konvolusi dan mengurangi ukuran data dari *layer* di bawahnya. Berbagi nilai bobot pada CNN penggunaannya akan bersamaan dengan penentuan skema *pooling* yang tepat bagi model. Khusus untuk CNN akan dipaparkan lebih lanjut pada subbab selanjutnya.

Selanjutnya untuk kelas ketiga yaitu *hybrid deep architecture* mengacu kepada teknik yang menggunakan arsitektur generatif sekaligus diskriminatif. Beberapa teknik dengan *hybrid deep architecture* di antaranya yang telah dipublikasikan oleh Hinton dan Salakhutidinov (2006). Pada penelitian tersebut di ajukan suatu teknik reduksi dimensi data menggunakan pendekatan *neural network*.

Algoritme CNN

Seperti telah disebutkan sebelumnya dua tahapan utama pada arsitektur CNN adalah tahapan *convolution* dan tahapan *subsampling (pooling)*. Pada Gambar 13 diilustrasikan ada sebuah *input* berupa gambar 1 *channel (grayscale image)* yang diinputkan pada CNN. Dimensi dari gambar tersebut adalah 32x32 pixels. Adapun

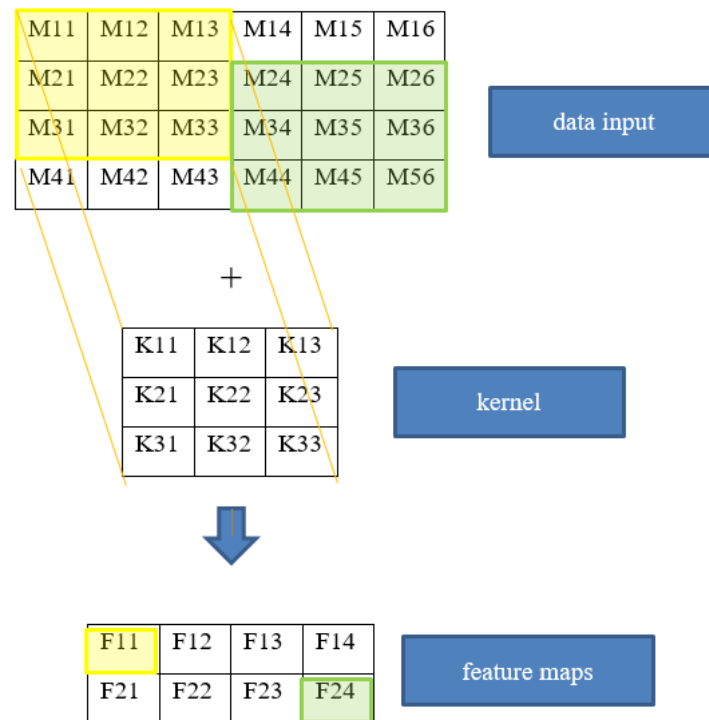
arsitektur CNN pada ilustrasi adalah 2 pasang *convolution-pooling* yang diikuti dua *fully connected layer*.



Gambar 2 Arsitektur CNN pada MNIST (Le Cun *et al.* 1998)

Penjelasan detail dari ilustrasi yang diberikan adalah:

- Pada tahap pertama *convolution* digunakan 6 buah kernel berukuran 5x5 pixels. Sebagai hasil dari tahapan *convolution* ini adalah C1 yang berupa 6 buah *feature Maps* berukuran 28x28 pixels. Mode pada tahapan ini digunakan *stride* 1 pixel dan mode *convolution*-nya valid (tanpa *padding*)
- Selanjutnya 6 *feature maps* yang berukuran 28x28 pixels ini, diterapkan *pooling* (*subsampling*) dengan faktor 2. Sebagai hasilnya setiap *feature maps* akan menjadi berukuran 14x14 pixels (S2).
- Tahap *convolution* kedua adalah penggunaan 16 kernel berukuran 5x5 pixels, dengan mode valid dan *stride* 1. Sebagai hasilnya adalah *feature maps* (C3) yang masing-masing ada 16 buah matriks dengan ukuran 10x10 pixels.
- Dilanjutkan dengan tahapan *subsampling* juga dengan faktor 2. Sebagai hasil dari tahapan ini adalah matriks yang masing-masing berukuran 5x5 pixels (S4).
- Selanjutnya keseluruhan nilai (neuron) pada *subsampling layer* yang terakhir akan di koneksikan kepada 120 neuron pada *layer fully connection*.
- Dilanjutkan dikoneksikan pada 84 neuron pada *layer fully connection* kedua.
- Dan pada *layer* terakhir dikoneksikan pada 10 neuron output.
- Catatan tambahan adalah bahwa titik antara *layer convolution* dengan *layer subsampling* di terapkan fungsi aktivasi (misalnya: tanh, sigmoid, ataupun ReLu)
- Fungsi aktivasi juga diterapkan baik antara *subsampling layer* yang terakhir dengan *fully connected layer* pertama, maupun di antara setiap pasangan *fully connected layer*.

Gambar 3 Ilustrasi *convolution*

Agar penjelasan lebih lengkap berikut ini adalah penjelasan tentang *convolution* dan *subsampling*. Misalkan diberikan sebuah gambar dengan dimensi berukuran 4x6 pixels dan sebuah kernel convolution berukuran 3x3 pixels (Gambar 14) maka akan dihasilkan perhitungan setiap nilai *feature maps* yang dapat dijelaskan sebagai berikut:

- Nilai F11 diperoleh ketika kernel di konvolusikan terhadap matriks input dengan posisi K22 seletak dengan M22.

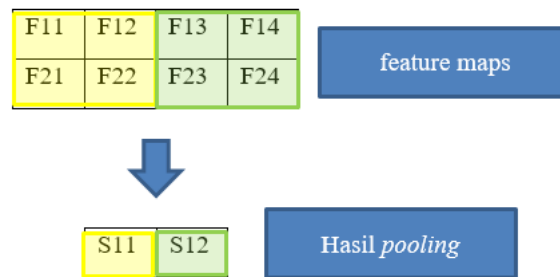
$$F11 = (M11 \times K11 + M12 \times K12 + M13 \times K13) + (M21 \times K21 + M22 \times K22 + M23 \times K23) + (M31 \times K31 + M32 \times K32 + M33 \times K33)$$
- Nilai F12 diperoleh ketika kernel di konvolusikan terhadap matriks input dengan posisi K22 seletak dengan M23 (area data input bergeser ke kanan satu pixel).

$$F12 = (M12 \times K11 + M13 \times K12 + M14 \times K13) + (M22 \times K21 + M23 \times K22 + M24 \times K23) + (M32 \times K31 + M33 \times K32 + M34 \times K33)$$
- Pergeseran ke kanan satu pixel tanpa padding mengakibatkan pergeseran ke kanan dari kernel hanya bisa dilakukan sebanyak 3 kali, sehingga secara total akan ada empat buah kolom nilai pada *feature maps*.
- Adapun pergeseran ke bawah sebanyak satu pixel tanpa *padding* mengakibatkan pergeseran ke bawah hanya bisa dilakukan sebanyak 1 kali, sehingga secara total secara baris hanya akan ada dua buah nilai *feature maps*.
- Dimensi feature maps adalah 2 baris x 3 kolom pixels.

- Nilai terakhir feature maps adalah F24 diperoleh ketika kernel di konvolusikan terhadap matriks input dengan posisi K22 seletak dengan M35

$$F24 = (M24 \times K11 + M25 \times K12 + M26 \times K13) + (M34 \times K21 + M35 \times K22 + M36 \times K23) + (M34 \times K31 + M35 \times K32 + M36 \times K33)$$

Selanjutnya tahapan subsampling diperoleh dengan mengambil nilai *summary* dari *feature maps*. Nilai *summary* dapat dihitung dari *average* ataupun cukup diambil nilai maksimum. Gambar 15 memberikan ilustrasi dari tahapan *subsampling* dengan faktor *pooling* = 2.



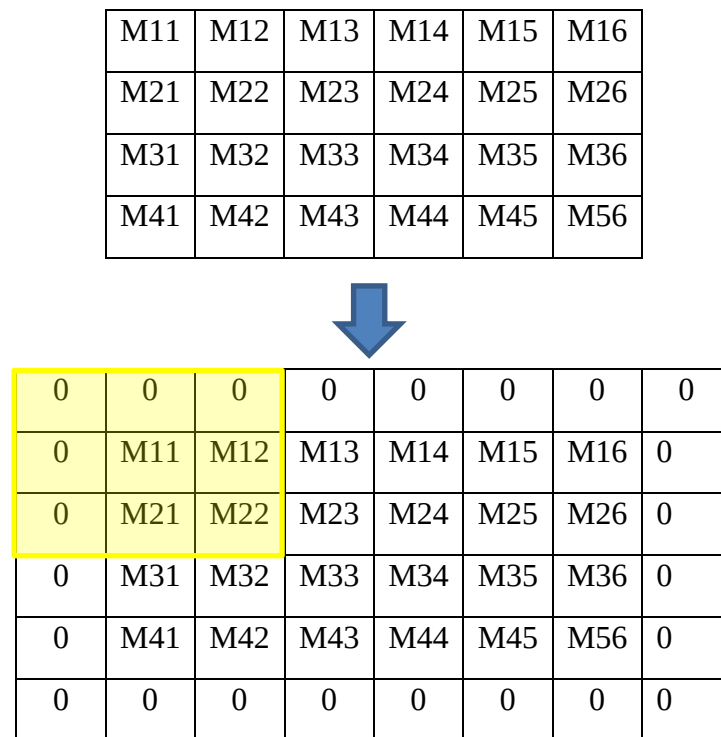
Gambar 4 Ilustrasi *subsampling*

Penentuan nilai hasil *subsampling* (*pooling*) dapat dijelaskan sebagai berikut:

- Jika yang digunakan adalah mode *average* maka nilai S11 diperoleh dari rata-rata F11, F12, F21 dan F22. Sehingga $S11 = (F11 + F12 + F21 + F22) / 4$
- Jika yang digunakan adalah mode *maximum* maka nilai S11 diperoleh dari nilai terbesar di antara F11, F12, F21 dan F22. Sehingga $S11 = \max(F11, F12, F21, F22)$.
- Aturan untuk S12 adalah sama, hanya saja data *feature maps* yang terlibat adalah F13, F14, F23 dan F24.

Variasi bagi tahapan *convolution* meliputi *stride* dan *padding*. *Stride* adalah lebar langkah pergeseran baik arah ke kanan ataupun ke bawah. Adapun *padding* adalah penambahan pixel pada tepi sedemikian sehingga setiap pojok data input dapat bertindak sebagai pusat *convolution* (pojok-pojoknya dapat seletak dengan *center* dari matriks kernel). Pada ilustrasi di Gambar 8 *center* dari kernel adalah posisi K22.

Gambar 16 memberikan ilustrasi dari pemberian *padding* pada matriks data *input* ketika ukuran kernel adalah 3x3 pixels. Dengan pemberian *padding* ini maka akibatnya hasil *feature maps* akan sama dengan ukuran data input (dengan catatan *stride* yang digunakan adalah pergeseran 1 pixels).

Gambar 5 Ilustrasi *padding*

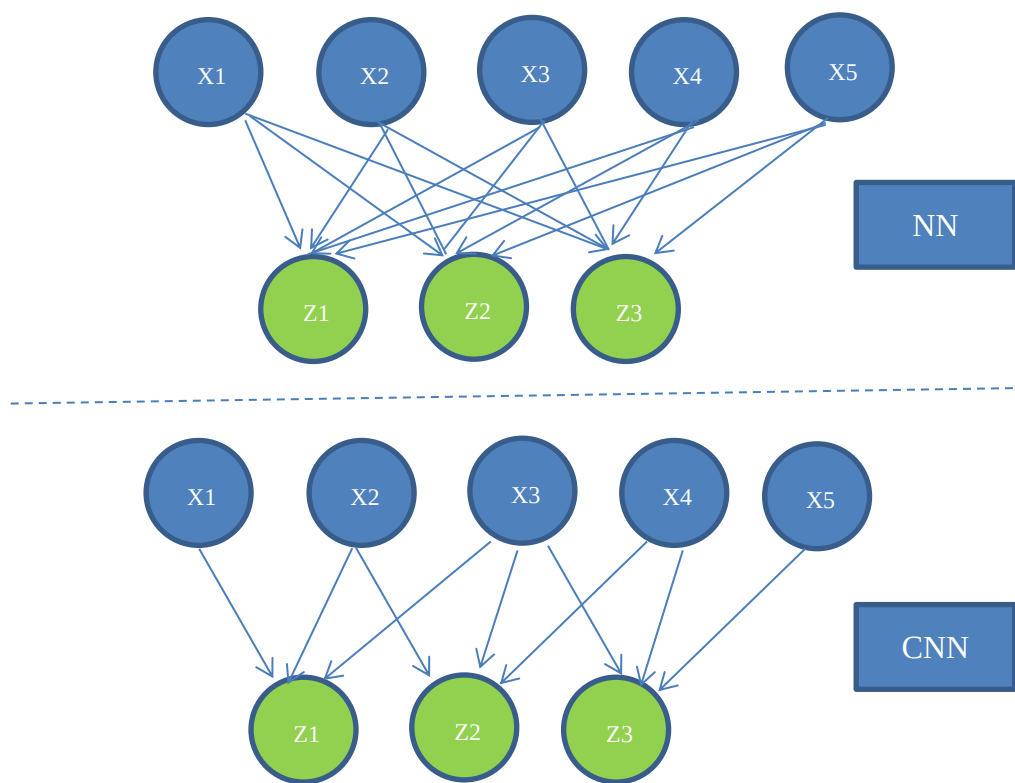
Selanjutnya Gambar 11 memberikan ilustrasi dari aksi *convolution* dengan *stride* berbeda. Dengan perbedaan *stride* tentunya jumlah pergeseran yang mungkin dilakukan akan berbeda pula.

Gambar 6 Ilustrasi *stride*

Semakin besar ukuran langkah *stride*, maka jumlah pergeseran akan menjadi lebih sedikit. Seperti pada Gambar 17 ini, ketika *stride* adalah satu langkah maka pergeseran arah kekanan yang mungkin adalah sebanyak 3 kali, sedangkan ketika *stride* bernilai dua langkah maka pergeseran ke kanan yang mungkin hanya satu kali saja.

Kelebihan CNN Dibandingkan *Neural Network*

Basis utama dari CNN adalah arsitektur Neural Network (NN), lalu apa kelebihan dari CNN dibandingkan NN. Lalu apa yang menjadi kelebihan CNN dibandingkan NN? Ada tiga sifat utama yang menjadi kelebihan CNN dibandingkan NN yaitu: *sparse connectivity*, *parameter sharing*, dan *equivariant*. Berikut adalah penjelasan yang disertai ilustrasi.

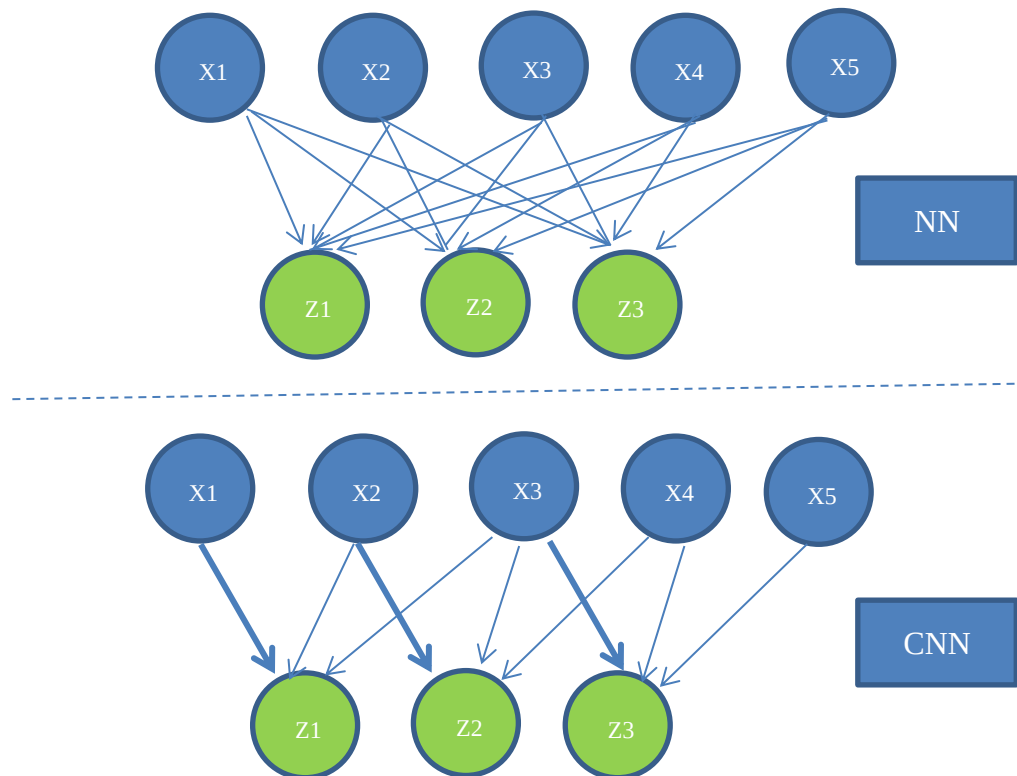


Gambar 7 Ilustrasi *sparse connectivity*

Sparse Connectivity - Pada arsitektur CNN konektifitas antar neuronnya relatif lebih sedikit jika dibandingkan dengan arsitektur NN yang *fully connected*. Perlu diingat bahwa setiap konektifitas bermakna bahwa ada satu parameter yang harus dilatih. Akibatnya pada kasus untuk ukuran neuron yang sama jumlahnya maka jumlah parameter yang dilatih pada CNN akan menjadi lebih sedikit. Secara komputasi hal ini akan menjadikan pelatihan pada arsitektur CNN akan menjadi lebih ringan dibandingkan NN. Gambar 18 memperlihatkan bahwa pada arsitektur NN setiap neuron (x_i , warna biru) pada *layer input* akan dilibatkan untuk menghitung setiap neuron pada *layer* berikutnya (setiap z_i , warna hijau). Berbeda halnya pada arsitektur CNN, pada kasus ini diumpamakan ukuran kernel adalah 1×3 ,

maka untuk menghitung sebuah neuron hijau (z_i) hanya akan dilibatkan tiga buah neuron biru dari *layer input*. Pada kasus ilustrasi ini parameter yang dilatih pada NN ada sebanyak 5×3 yaitu 15 buah sedangkan pada CNN hanya 3×3 yaitu 9 buah saja.

Parameter sharing - Pada CNN setiap parameter pada kernel adalah parameter yang digunakan secara berulang, dilatih berulang untuk beberapa tipe data yang berbeda. Pada kondisi tersebut sebuah parameter di-*sharing*, pemanfaatannya. Berikut adalah penjelasan disertai ilustrasinya (Gambar 19). Pada NN setiap koneksi adalah parameter berbeda sedangkan pada CNN sebagai contoh saja tiga buah koneksi yang tebal adalah parameter kernel yang sama, namun digunakan oleh input data berbeda untuk menghitung neuron hijau yang berbeda pula. Dengan parameter sharing maka proses pelatihan parameter dibuat adaptif pada beberapa *neuron* input sekaligus.



Gambar 8 Ilustrasi *parameter sharing*

Equivariant – Sederhananya adalah bahwa jika kita menerapkan sebuah transformasi terhadap data input (katakan transformasi X) lalu setelah itu diterapkan *convolution* dan menghasilkan $Z1$. Maka dengan urutan yang sebaliknya, yaitu misalkan data input kita *convolution* dulu lalu hasilnya baru diterapkan transformasi X dan menghasilkan nilai $Z2$. Maka dengan konsep *equivariant* nilai $Z1$ dan $Z2$ akan dijamin merupakan nilai yang sama.

Formulasi Matematika untuk CNN

Operasi Konvolusi

Untuk memudahkan mari kita tinjau sebuah data sinyal 1-D dengan ukuran $1 \times n$, $X = [x_1, x_2, x_3, \dots, x_n]$. Sebuah kernel dengan ukuran $1 \times m$, $K = [k_1, k_2, k_3, \dots, k_m]$, dengan $n > m$. Output Y sebagai hasil konvolusi dapat dinotasikan sebagai $Y = X * K$. Jika mode konvolusi adalah valid (tanpa *padding*) dan *stride* bernilai 1 maka dimensi Y adalah $1 \times (n - m + 1)$. $Y = [y_1, y_2, y_3, \dots, y_{n-m+1}]$

Data input:

$$X = [x_1, x_2, x_3, \dots, x_n]$$

Filter Kernel:

$$K = [k_1, k_2, k_3, \dots, k_m]$$

Output hasil konvolusi:

$$Y = [y_1, y_2, y_3, \dots, y_{n-m+1}]$$

Notasi konvolusi menggunakan *:

$$Y = X * K$$

Maka

$$y_1 = x_1 \cdot k_1 + x_2 \cdot k_2 + \dots + x_m \cdot k_m$$

$$y_2 = x_2 \cdot k_1 + x_3 \cdot k_2 + \dots + x_{m+1} \cdot k_m$$

$$y_3 = x_3 \cdot k_1 + x_4 \cdot k_2 + \dots + x_{m+2} \cdot k_m$$

$$y_i = x_i \cdot k_1 + x_{i+1} \cdot k_2 + \dots + x_{m+i-1} \cdot k_m$$

$$y_i = \sum_{j=1}^m x_{j+i-1} \cdot k_j$$

Dengan catatan bahwa untuk setiap j dan i harus dipenuhi $(j + i - 1) \leq n$

Operasi Konvolusi, Aktivasi dan Pooling

Katakan operasi konvolusi adalah fungsi $C(X)$, Aktivasi adalah fungsi $A(X)$, dan Pooling adalah $P(X)$. Maka untuk seperangkat operasi konvolusi, aktivasi, dan Pooling secara berurutan terhadap X dan menghasilkan Y dapat dituliskan sebagai fungsi komposisi berikut:

$$Y = P \left(A \left(C(X) \right) \right) = (P \circ A \circ C)(X)$$

Operasi Pooling dengan factor skala = $1/m$ dengan m adalah faktor dari n

Katakan data sebelum *sampling* adalah

$$X = [x_1, x_2, x_3, \dots, x_n]$$

Maka output Y akan berdimensi $1 \times \left(\frac{m}{n}\right)$

$$Y = [y_1, y_2, y_3, \dots, y_{\frac{m}{n}}]$$

Seandainya fungsi pooling adalah maksimum maka

$$y_1 = \max(x_1, x_2, x_3, \dots, x_m)$$

$$y_2 = \max(x_{m+1}, x_{m+2}, x_{m+3}, \dots, x_{2m})$$

$$y_i = \max(x_{(i-1)m+1}, x_{(i-1)m+2}, x_{(i-1)m+3}, \dots, x_{im}), \text{ dengan } i \leq \frac{m}{n}$$

2 PENERAPAN CNN UNTUK BERAGAM KASUS KLASIFIKASI

Teknik CNN pada Klasifikasi Berbasis *Feature* Akustik

Telah disebutkan sebelumnya bahwa *feature* bagi identifikasi kematangan buah melon secara umum berupa data akustik dan data visual. Data akustik yang digunakan merupakan data rekaman suara ketukkan bandul kepada buah melon. Sedangkan data visual yang digunakan dapat berupa gambar digital dari buah melon. Pada bagian ini akan dipaparkan beberapa catatan keberhasilan model klasifikasi berbasis CNN yang memiliki hasil baik untuk kasus klasifikasi dengan *feature* berbasis suara dan citra dari penelitian-penelitian sebelumnya.

1. Penelitian pengenalan suara (*speech recognition*) memberikan hasil bahwa CNN yang diterapkan pada *hybrid neural network – hidden markov model* bagi pengenalan suara dapat meningkatkan akurasi model yang dibentuk. Dengan CNN nilai akurasi mencapai 79,93%. Hasil tersebut lebih baik dari *neural network* standar (77,05%) ataupun bahkan *neural network* yang sudah dilakukan *pretraining* oleh *Deep Belief Network* (79,5%). Pada penelitian tersebut data yang digunakan adalah *TIMIT dataset* (Hamid *et al.* 2012).
2. Penelitian klasifikasi fonem pada sinyal suara membuktikan bahwa arsitektur CNN mendapatkan nilai akurasi lebih baik dibandingkan arsitektur *Multi Layer Perceptron* (MLP) standar. Menggunakan data mentah sinyal suara, model MLP hanya mendapatkan tingkat akurasi 38,91% di sisi lain dengan CNN tingkat akurasi dapat mencapai 67,88% (Palaz *et al.* 2013).

3. Penelitian pendeteksian *onset* pada sinyal musik mendapatkan hasil bahwa CNN lebih baik dari *Recurrent Neural Network* (RNN) yang sebelumnya bertindak sebagai model pendeteksi terbaik. Pada musik *onset* dapat didefinisikan sebagai awal nada musik ataupun awal dari suara lainnya, ketika amplitudo naik dari nol ke titik puncak inisial. Menggunakan RNN tingkat akurasi model adalah 89,2% sedangkan menggunakan CNN dapat mencapai 91,7% (Schluter & Bock 2014).
4. Kembali terkait kesuksesan CNN pada penelitian di bidang pengenalan suara, penerapan CNN terbukti dapat mengurangi nilai *error* dari model pengenalan suara standar berbasis *Deep Neural Network* (DNN)- *Hidden Markov Model* (HMM). Menggunakan data set TIMIT pada eksperimen, tingkat *error* yang dapat diturunkan mencapai 6% - 10% (Hamid *et al.* 2014).
5. Klasifikasi suara menggunakan CNN dan Tensor Deep Stacking Network (TDSN). Penelitian ini menggunakan dua buah dataset: (1) Environment Sound Classification (ESC)-10 yang memiliki 400 data, dengan 10 kategori; dan (2) ESC-50 yang memiliki 2000 data suara dengan 50 kategori kelas. Pada penelitian ini Gambar spectrogram dari suara diekstraksi. Spectrogram yang dihasilkan diekstraksi menggunakan fungsi `spectrogram()` pada MATLAB. Sebagai hasilnya adalah model klasifikasi dari kombinasi CNN memperoleh akurasi 77.85% (untuk data ESC-10) dan 49 % untuk dataset ESC-50)). Sedangkan untuk model klasifikasi yang dibentuk TDSN yang hanya menggunakan dataset ESC-10 hanya diperoleh nilai akurasi 56% (Khamparia *et al.* 2017).
6. Klasifikasi bunyi menggunakan CNN. Datanya berupa 6.776 spectrograms dari data bunyi: gergaji mesin, kicau burung, lolongan monyet, dan suara-suara lainnya yang lazim terdengar di hutan. Dataset tersebut dibuat mandiri, hasil mengumpulkan dari internet. Spectrograms yang diekstraksi memanfaatkan fungsi `matplotlib`. Menggunakan CNN dengan fungsi Aktivasi ReLU, dan pada output *layer* menggunakan sigmoid diperoleh akurasi 85% (Jaiswal & Patel 2018).
7. Pendekatan CNN juga telah digunakan pada permasalahan klasifikasi suara di lingkungan konstruksi. Dataset yang digunakan adalah ESC dan suara beberapa peralatan konstruksi dari beragam merk dan tipe. Pada penelitian ini digunakan 5 kategori kelas. Feature yang digunakan adalah Mel-Spectrogram. Mel spectrogram adalah spectrogram dari suara yang frekuensinya telah dikonversi ke skala mel. CNN dan 4 algoritme pembandingnya digunakan pada penelitian ini: Random Forest, Multi Layer Perceptron (MLP), k-NN, dan SVM. Akurasi untuk setiap teknik klasifikasi secara beturut-turut adalah sebagai berikut: CNN (.97), Random Forest (.93), MLP (.92), k-NN (.85), dan SVM (.84). Dari hasil ini CNN berhasil membentuk model klasifikasi dengan akurasi terbaik (Maccagno *et al.* 2020).

Teknik CNN pada Klasifikasi Berbasis *Feature Visual*

Selanjutnya untuk permasalahan klasifikasi menggunakan *feature* berbasis *image* CNN telah berhasil mencatatkan beberapa kesuksesan yang di antaranya adalah

1. CNN berhasil sdigunakan untuk mengklasifikasikan gambar rambu lalu lintas dan lampu lalu lintas. Ada empat kelas yan digunakan: tanda stop, tanda dilarang stop, lampu hijau dan lampu merah. Total data sebanyak 360 buah gambar. Menggunakan *Raw feature RGB-D images* (langsung pixel nya) teknik klasifikasi diterapkan. Sebagai basis arsitektur model klasifikasi digunakan arsitektur CNN Alexnet untuk *training* (pelatihan). Selanjutnya dengan teknik CNN dan *fine tuning* data diklasifikasi dan memperoleh akurasi 93.33% (Jmour, Zayen, & Abdelkrim 2018).
2. Pada penelitian dengan judul *Pure-CNN: A Framework for Fruit Images Classification* telah dikerjakan. Data berupa 55.244 gambar berwarna dari buah-buahan, dengan 81 buah kategori buah berbeda digunakan. Sebagai *feature* adalah *pixels images (raw data)* dengan dimensi 100 pixels x100 pixels. Pure CNN dengan 7 buah layer CNN diujikan dan dengan menggunakan global average pooling akurasi klasifikasi yang diperoleh mencapai 98.88% (Kausar *et al.* 2018).
3. Catatan keberhasilan penggunaan CNN juga diperoleh ketika CNN digunakan untuk mengklasifikasikan 1,2 juta buah gambar (*image*) *high-resolution* dengan 1000 kelas kategori. Data tersebut diambil dari dataset ImageNet Large Scale Visual Recognition Competition (ILSVRC). Feature yang digunakan adalah *raw data image* dari posisi tengah yang di *downsampling* ke ukuran 256 x 256 pixels. Dengan mengkombinasikan CNN model: GoogLeNet dan ResNetDiperoleh *error* akurasi yang diperoleh sebesar 16,88% (Sultana, Sufian, & Duta 2018).
4. Masih menggunakan data citra dari buah-buahan. Menggunakan Dataset Fruits-360 yang terdiri atas 74 jenis gambar buah- buahan, dengan total 49.561 buah gambar. Terhadap *raw images pixels* (100x100 pixels) langsung diterapkan arsitektur CNN Alex-net yang dimodifikasi dan diperoleh akurasi 98% (Zu *et al.* 2019).

3 IMPLEMENTASI CNN DENGAN CONTOH EKSPERIMEN

Telah disebutkan sebelumnya bahwa pada pendekatan *deep learning*, sebuah algoritma akan berperan pada dua fungsi, yaitu fungsi sebagai ekstraktor *feature* dan juga sebagai klasifier. Pada eksperimen yang telah dilakukan dan akan dilaporkan pada dokumen ini juga digunakan skema tersebut. Data *image* yang digunakan adalah data yang sama namun tidak mengalami tahapan *feature extraction*. Teknik *deep learning* yang digunakan pada eksperimen ini adalah *Convolutional Neural Network* (CNN). Pada CNN tahapan ekstraksi *feature* diperankan oleh tahapan konvolusi dan pooling. Konvolusi bertindak mengekstraksi *feature* lokal pada sebuah pixel dan pixel-pixel tetangganya yang berdekatan, sedangkan tahapan pooling berperan merangkum informasi terpenting dari setiap tahapan konvolusi yang dilakukan.

Contoh Data Eksperimen

Salah satu kebutuhan pembentukan model dengan pendekatan *deep learning* adalah jumlah ukuran data yang harus banyak. Sedangkan pada kondisi saat ini, jumlah data yang telah dimiliki hanya 220 data secara total. Atas kondisi tersebut maka untuk mencobakan CNN pada eksperimen pendahuluan ini telah digunakan juga dataset pada kasus berbeda. Kasus dataset yang digunakan adalah data set klasifikasi COVID19 berdasarkan image CT-Chest. Data yang digunakan pada eksperimen ini merupakan data yang berasal dari *open dataset* dan dapat diperoleh dari <https://github.com/UCSD-AI4H/COVID-CT>. Gambaran umum tentang dataset ini adalah:

- Data *image* berasal dari beragam alat CT-Scan, sehingga ukuran gambarnya sangat beragam.
- Untuk menyeragamkan ukuran maka setiap gambar akan di-*resize* sehingga setiap gambar berdimensi 224x224 pixels (Gambar 3.19).
- Data *image* total ada sebanyak 746 buah (2 kelas): terdiri atas 349 covid *image* dan 397 noncovid *image*.
- Skenario uji tidak menggunakan skenario cross-validation, namun data hanya dibagi dua untuk *training* dan untuk validasi. Sebanyak 20% data (149 data: 70 covid, 79 noncovid) digunakan sebagai data validasi dan 80% sisanya (597 data: 279 covid, 318 noncovid) digunakan sebagai data *training*.
- Setiap pengujian akan diulangi 3 kali (*epoch* bagi *training* adalah 50).



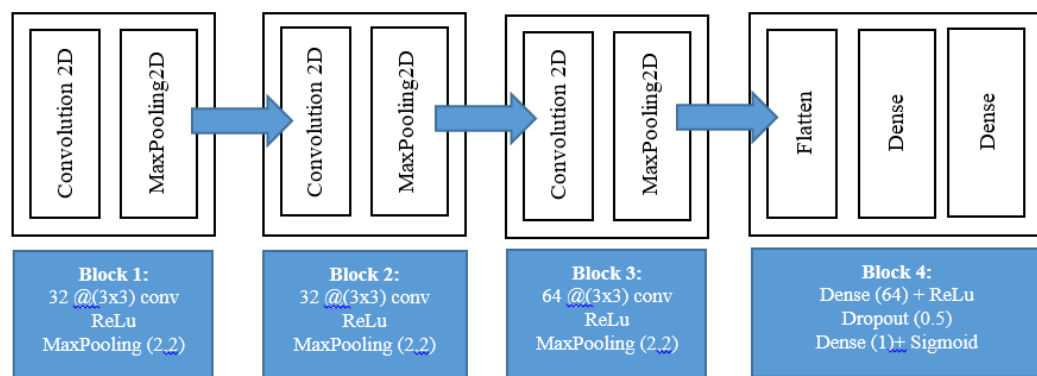
Gambar 9 Contoh data *image* dari dataset CT-chest

Arsitektur CNN yang digunakan merupakan arsitektur CNN seperti pada <https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html>. Arsitektur ini merupakan arsitektur ringan yang terinspirasi Deep CNN VGG16 (Gambar 3.20). Pada arsitektur ini diterapkan teknik *bottleneck features* dengan cara menampung *output* pada dua buah numpy array dan menerapkan teknik aktivasi sebelum tahapan pooling untuk kemudian dilatih pada *fully-connected layer* dengan jumlah neuron yang sedikit. Gambar 3.21 memberikan kode python sebagai representasi arsitektur ini. Arsitektur disusun oleh 3 blok pasangan konvolusi dan pooling serta satu set blok *fully connected layer* di bagian akhir. Menggunakan arsitektur ini telah dilaporkan dapat mengklasifikasi 1000 buah *image* kasus *binary class* hanya dalam waktu satu menit dengan akurasi

di atas 90% (epoch training = 50). Kasus yang digunakan adalah *image* kucing dan anjing yang diperoleh dari dataset *Image net*. Pada Eksperimen kami menggunakan python dengan *library* keras untuk mengimplementasikan model arsitektur ini.

Arsitektur CNN yang Digunakan

Arsitektur CNN yang digunakan, ketika membentuk model cenderung memiliki nilai loss dan akurasi terhadap data training yang fluktuatif pada setiap *epoch*. Jika kita mengacu kepada output proses tahapan *training* (Gambar 3.22) maka nilai akurasi evaluasi pernah mencapai nilai 0,764 untuk kemudian menurun dan di akhir epoch nilai tertinggi akurasi yang diperoleh adalah 0,758. Secara berturut-turut nilai akurasi untuk 3 kali ulangan mendapatkan masing-masing nilai akurasi: 0,7181, **0,7584** dan 0,7181. Nilai parameter kinerja klasifikasi khusus pada model dengan nilai akurasi tertinggi (pengukuran ulangan ke 2) adalah: *precision*: 0,760, *recall*: 0,767, *F-score*: 0,758. Adapun *confusion matrix* bagi model klasifikasi ini diberikan pada Tabel 3.1. Dari *confusion matrix* terlihat bahwa nilai *false-negatif* dari model mencapai 22 buah (lebih dari 14%). Maknanya adalah bahwa model yang terbentuk relatif sulit untuk memprediksi kelas negatif covid (normal) kepada kelasnya yang benar. Gambar 3.23 memberikan contoh *image* data normal yang salah diprediksi oleh model. Terlihat bahwa memang dari tampilan data pada *image* tersebut area tengah dari *image* didominasi warna terang yang seolah seperti pengkabutan pada area paru-paru. Kondisi inilah yang diduga menjadikan model terkelabui dan salah memprediksi.



Gambar 10 Arsitektur CNN yang diterapkan

```

23 #Arsitektur Ars-01
24 #Blok 1
25 model = Sequential()
26 model.add(Conv2D(32, (3, 3), input_shape=input_shape))
27 model.add(Activation('relu'))
28 model.add(MaxPooling2D(pool_size=(2, 2)))
29 #Blok 2
30 model.add(Conv2D(32, (3, 3)))
31 model.add(Activation('relu'))
32 model.add(MaxPooling2D(pool_size=(2, 2)))
33 #Blok 3
34 model.add(Conv2D(64, (3, 3)))
35 model.add(Activation('relu'))
36 model.add(MaxPooling2D(pool_size=(2, 2)))
37 #Blok 4
38 model.add(Flatten())
39 model.add(Dense(64))
40 model.add(Activation('relu'))
41 model.add(Dropout(0.5))
42 model.add(Dense(1))
43 model.add(Activation('sigmoid'))

```

Gambar 11 Implementasi CNN dengan Keras.

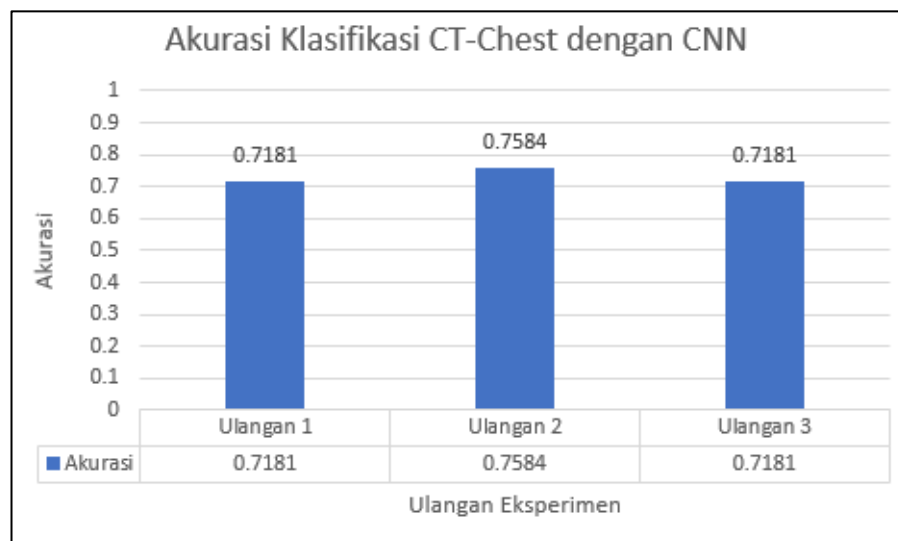
Hasil Eksperimen

```

1879 35/37 [=====>...] - ETA: 1s - loss: 0.3115 - accuracy: 0.8571
1880 36/37 [=====>...] - ETA: 0s - loss: 0.3252 - accuracy: 0.8524
1881 37/37 [=====] - ETA: 0s - loss: 0.3238 - accuracy: 0.8530
1882 37/37 [=====] - 29s 789ms/step - loss: 0.3238 - accuracy: 0.8530 - val_loss: 0.9332 -
    val_accuracy: 0.7639
1883 Epoch 48/50
1884
1885 1/37 [.....] - ETA: 0s - loss: 0.1958 - accuracy: 0.8750
1886 2/37 [>.....] - ETA: 13s - loss: 0.2028 - accuracy: 0.8750
1887 3/37 [=.....] - ETA: 18s - loss: 0.2035 - accuracy: 0.8958

```

Gambar 12 Output dari tahapan pembentukan model oleh CNN



Gambar 3.23 Akurasi klasifikasi CT-Chest dengan CNN

Gambar 13 Confusion matrix bagi model klasifikasi CNN dengan nilai akurasi tertinggi (model pada ulangan ke-2)

Aktual	Prediksi		Total
	noncovid	covid	
noncovid	57	22	79
covid	14	56	70
Total	71	78	149



Gambar 14 Data *image* CT-Chest yang salah diprediksi oleh model

4 KESIMPULAN

Dua tahapan utama pada arsitektur CNN adalah tahapan *convolution* dan tahapan *subsampling* (*pooling*). Dalam *machine learning*, CNN merupakan teknik yang termasuk pada *feed-forward neural network*. Arsitektur CNN setiap individu *neuron* disusun sedemikian sehingga dapat memberikan respon terhadap *region* yang saling *overlapping* pada suatu area visual (Ciresan 2011). CNNs merupakan *hierarchical neural networks* yang tersusun atas beberapa *convolutional layer* dan beberapa *subsampling layer*. Pada pemanfaatannya CNN dapat divariasikan pada bagian jenis dan jumlah dari *convolutional layer* dan *subsampling layer* yang dimilikinya. Arsitektur CNN pada penerapannya bertindak sebagai modul ekstraksi *feature* sekaligus juga sebagai modul klasifier. Berdasarkan hasil kajian studi literatu CNN berhasil menjadi state of the art pada banyak kasus permasalahan klasifikasi.

DAFTAR PUSTAKA

Ciresan CD, et al. 2011. Flexible, High Performance Convolutional Neural Networks for Image Classification. Proceedings of the Twenty-Second

- International Joint Conference on Artificial Intelligence (IJCAI'11), Vol. Two, Pages 1237-1242.
- Dey S. 2018. CNN application on structured data-Automated Feature Extraction.[diunduh 1 Nov 2020] Tersedia pada <https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e>
- Doshi K. 2020. Audio Deep Learning Made Simple (Part 1): State-of-the-Art Techniques.<https://towardsdatascience.com/audio-deep-learning-made-simple-part-1-state-of-the-art-techniques-da1d3dff2504>. Diakses: 30 Maret 2021.
- Giri EP, *et al.* 2016. Ischemic Stroke Identification Based on EEG and 1D Convolutional Neural Network. Proceeding International Conference on Advanced Computer Science and Information Systems (ICACSIS) 2016.
- Hinton GE & Salakhutdinov RR. 2006. Reducing the Dimensionality of Data with Neural Networks. Science 28 Jul 2006: Vol. 313, Issue 5786, pp. 504-507 DOI: 10.1126/science.1127647.
- Hamid A S, Mohamed A, Jiang H, & Pen G. 2012. Applying Convolutional Neural Networks Concepts to Hybrid NN-HMM Model for Speech Recognition. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 4277-4280 25-30 March 2012, Kyoto, Japan.
- Hamid AS, Mohamed AR, Jiang H, Deng L, Penn G & Yu D. 2014. Convolutional Neural Networks for Speech Recognition. IEEE/ACM Transactions On Audio, Speech, And Language Processing, VOL. 22, NO. 10, October 2014.
- Hortikultura Kementan. 2019. Si Melon dari Serang.[diunduh 1 Okt 2020] Tersedia pada: <http://hortikultura.pertanian.go.id/?p=3169#:~:text=Sentra%20produksinya%20tersebar%20di%20berbagai,Varietas%20melon%20yang%20dikembangkan%20pun>
- Jaiswal K and Patel DK. 2018. Sound Classification Using Convolutional Neural Networks. 2018 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM), Bangalore, India, 2018, pp. 81-84, doi: 10.1109/CCEM.2018.00021.
- Jmour N, Zayen S, and Abdelkrim A. 2018. Convolutional neural networks for image classification. 2018 International Conference on Advanced Systems and Electric Technologies (IC_ASET), Hammamet, 2018, pp. 397-402, doi: 10.1109/ASET.2018.8379889.
- Khamparia A, Gupta D, Nguyen NG, Khanna A, Pandey B and Tiwari P. 2019. Sound Classification Using Convolutional Neural Network and Tensor Deep Stacking Network," in IEEE Access, vol. 7, pp. 7717-7727, 2019, doi: 10.1109/ACCESS.2018.2888882.
- Lecun Y, Bottou Y, Bengio Y and Haffner P. 1998. "Gradient-based learning applied to document recognition," in Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998, doi: 10.1109/5.726791.
- Li D. 2014. Overview paper: A tutorial survey of architectures, algorithms, and applications for deep learning. SIP (2014), vol. 3, e2, page 1 of 2<http://creativecommons.org/licenses/by/3.0/>, doi:10.1017/ATSIP.2013.99.
- Maccagno A, Mastropietro A, Mazziotta U, Scarpiniti M, Lee YC, and Uncini A. 2020. A CNN Approach for Audio Classification in Construction Sites. In:

- Esposito A., Faundez-Zanuy M., Morabito F., Pasero E. (eds) *Progresses in Artificial Intelligence and Neural Systems. Smart Innovation, Systems and Technologies*, vol 184. Springer, Singapore. https://doi.org/10.1007/978-981-15-5093-5_33
- Palaz D, Collobert R, Doss MM. 2013. Estimating Phoneme Class Conditional Probabilities from Raw Speech Signal using Convolutional Neural Networks. arXiv preprint arXiv:1304.1018.
- Schlüter J & Böck S. 2014. Improved Musical Onset Detection With Convolutional Neural Networks. *Proceedings of the 2014 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP 2014)*.
- Sultana F, Sufian A, Dutta P. 2018. Advancements in Image Classification using Convolutional Neural Network. 2018 Fourth International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN), Kolkata, India, 2018, pp. 122-129, doi: 10.1109/ICRCICN.2018.8718718.
- Tiffany FL, 2016. Teknik Budidaya Melon (Cucumis melo. L) secara Tabulampot di Kebun Buah Mekarsari, Cileungsi, Jawa Barat. Laporan Praktek Kerja Lapangan (Techical Report): Departemen Biologi, FMIPA IPB.
- Zheng Y, Liu Q, Chen E, Ge Y & Zhao JL. 2014. Time Series Classification Using Multi-Channels Deep Convolutional Neural Networks. *Lecture Notes in Computer Science*, Volume 8485, pp 298-310.

